

DIABITIES DETECTION BINARY CLASSIFIRE USING TENSORFLOW

Dataset: Pima Indians Diabetes Dataset

Tool: Google Colab

Framework: TensorFlow

Objective: The objective of this project is to develop a binary classification model using TensorFlow to predict whether a patient has diabetes or not. Two models are developed and evaluated using the Pima Indians Diabetes Dataset.

Dataset Description: The Pima Indians Diabetes Dataset is used for diabetes prediction. The dataset contains 8 input features and one target variable called Outcome.

Input Features:

- * Pregnancies
- * Glucose
- * BloodPressure
- * SkinThickness
- * Insulin
- * BMI
- * DiabetesPedigreeFunction
- * Age

Target: Outcome

0 = No Diabetes

1 = Diabetes

1. Implement a diabetes detection binary classifier using Tensorflow. You can take any diabetes data from Kaggle. Note that there shouldn't be any hidden layer.

```
url = "https://raw.githubusercontent.com/npradaschnor/Pima-Indians-Diabetes-Dataset/master/diabetes.csv"
df = pd.read_csv(url)
df.head()
```

```
...      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0             6       148           72           35          0  33.6              0.627    50           1
1             1        85           66           29          0  26.6              0.351    31           0
2             8       183           64           0           0  23.3              0.672    32           1
3             1        89           66           23          94  28.1              0.167    21           0
4             0       137           40           35         168  43.1              2.288    33           1
```

```
print (df.shape)
print (df.isnull().sum())
```

```
(768, 9)
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI               0
DiabetesPedigreeFunction  0
Age               0
Outcome           0
dtype: int64
```

```
print (df.shape)
print (df.isnull().sum())
```

```
... (768, 9)
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI               0
DiabetesPedigreeFunction  0
Age               0
Outcome           0
dtype: int64
```



```
#Not hidden layer model
model1 = tf.keras.Sequential([
    tf.keras.layers.Dense(1, activation='sigmoid',input_shape=(8,))
])
model1.summary()
```

```
... /usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106:
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 1)	9

```
Total params: 9 (36.00 B)
Trainable params: 9 (36.00 B)
Non-trainable params: 0 (0.00 B)
```

1. Modify the above classifier given in above question by increasing one hidden layer and on dropout la

```
loss1, accuracy1 = model1.evaluate(X_test, y_test)
print("Model 1 Test Accuracy:", accuracy1)
```

5/5 ————— 0s 10ms/step - accuracy: 0.7013 - loss: 0.6636
Model 1 Test Accuracy: 0.701298713684082

```
model2 = tf.keras.Sequential([
    tf.keras.layers.Dense(16, activation='relu', input_shape=(8,)),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model2.summary()
```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_4"

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 16)	144
dropout_1 (Dropout)	(None, 16)	0
dense_9 (Dense)	(None, 1)	17

Total params: 161 (644.00 B)
Trainable params: 161 (644.00 B)
Non-trainable params: 0 (0.00 B)

```
loss2=accuracy2=model2.evaluate(X_test, y_test)
print("Model 2 Test Accuracy:", accuracy2)
```

5/5 ————— 0s 6ms/step - accuracy: 0.6948 - loss: 0.6278
Model 2 Test Accuracy: [0.6278437376022339, 0.6948052048683167]

```
predictions = model2.predict(X_test)
predictions = (predictions > 0.5).astype(int)
```

```
print("First 10 prediction:")
print(predictions[:10])
```

5/5 ————— 0s 8ms/step

First 10 prediction:

```
[[0]
 [0]
 [0]
 [0]
 [0]
 [0]
 [0]
 [0]
 [0]
 [0]]
```